

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

MACHINE LEARNING LABORATORY

Experiment 1: Decision Tree Classification

Experiment 2: Reinforcement Learning – Q-Learning

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and scikit-learn.

(a) Dataset Loading, Pre-processing and Train/Test Split

The Iris dataset contains 150 samples, four numerical features, and three classes: setosa, versicolor, and virginica. The dataset was checked for missing values; none were present. Since all input features are numerical, categorical encoding was not required. The data was split into 80% training and 20% testing data using stratification.

First 5 rows:

Sepal Length	Sepal Width	Petal Length	Petal Width	Target	Data Type
5.1	3.5	1.4	0.2	setosa	float64 / object
4.9	3.0	1.4	0.2	setosa	float64 / object
4.7	3.2	1.3	0.2	setosa	float64 / object
4.6	3.1	1.5	0.2	setosa	float64 / object
5.0	3.6	1.4	0.2	setosa	float64 / object

Missing-value check: all feature columns contain 0 missing values.

(b) Decision Tree Construction and Root Node

A Decision Tree Classifier was trained using the Gini Index as the splitting criterion. The tree recursively chooses the feature and threshold that provide the best class separation.

Python model: `DecisionTreeClassifier(criterion='gini', random_state=42)`

Root node: petal length (cm) ≤ 2.450

Justification: the root node is the first split selected by the trained tree because it produces the largest impurity reduction among the candidate splits under the Gini criterion. For this training split, petal length is selected as the root feature.

Tree structure:

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |   |--- petal length (cm) > 4.95
|   |   |       |--- sepal length (cm) <= 6.15
```

```

| | | | |--- sepal width (cm) <= 2.45
| | | | | |--- class: virginica
| | | | |--- sepal width (cm) > 2.45
| | | | | |--- class: versicolor
| | | |--- sepal length (cm) > 6.15
| | | | |--- class: virginica
| |--- petal width (cm) > 1.65
| | |--- petal length (cm) <= 4.85
| | | |--- sepal width (cm) <= 3.00
| | | | |--- class: virginica
| | | |--- sepal width (cm) > 3.00
| | | | |--- class: versicolor
| | |--- petal length (cm) > 4.85
| | | |--- class: virginica

```

(c) Model Evaluation

Accuracy: 0.9333 (93.33%)

Confusion Matrix (rows = actual, columns = predicted; order: setosa, versicolor, virginica):

Actual \ Predicted	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	1
virginica	0	1	9

Classification Report:

```

precision  recall  f1-score  support

setosa      1.00    1.00    1.00     10
versicolor  0.90    0.90    0.90     10
virginica   0.90    0.90    0.90     10

accuracy                0.93     30
macro avg    0.93    0.93    0.93     30
weighted avg  0.93    0.93    0.93     30

```

Performance comment: The model performs very well on the Iris dataset. The accuracy is high and most observations are classified correctly. The small number of errors occurs between the visually similar versicolor and virginica classes.

Two misclassified instances:

Index	Sepal L	Sepal W	Petal L	Petal W	Actual	Predicted
134	6.1	2.6	5.6	1.4	virginica	versicolor
77	6.7	3.0	5.0	1.7	versicolor	virginica

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective: Implement Q-Learning for a simple 4x4 Grid World and observe the agent's learned policy.

(a) Environment, Rewards, Q-table and Hyperparameters

The environment is a 4x4 grid with states 0–15. The agent starts at state 0 and the goal is state 15. Obstacles are placed at states 5 and 9. The available actions are Up (U), Down (D), Left (L), and Right (R).

Event	Reward	Meaning
Reach goal	+10	Episode terminates
Normal step	-1	Encourages shorter paths
Obstacle	-5	Discourages obstacle movement

Initial Q-table: a 16×4 matrix initialized to zeros.

Hyperparameters: α (learning rate) = 0.8, γ (discount factor) = 0.95, ϵ (exploration rate) = 0.20.

Q-learning update: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ for non-terminal transitions. For the terminal goal transition, the future-value term is zero.

(b) Training for 100 Episodes and Q-table Evolution

The algorithm was run for 100 episodes. An ϵ -greedy strategy was used so the agent occasionally explores random actions and otherwise selects the action with the highest Q-value.

Q-values after Episode 1 for representative states 0 and 10:

State	Up	Down	Left	Right
0	-0.800	-0.800	-0.800	-0.800
10	0.000	-0.800	0.000	0.000

Q-values after Episode 50 for representative states 0 and 10:

State	Up	Down	Left	Right
0	1.826	3.213	0.904	0.804
10	-1.530	8.128	-0.037	-0.960

Q-values after Episode 100 for representative states 0 and 10:

State	Up	Down	Left	Right
0	2.053	3.213	2.007	0.914
10	-1.530	8.499	-0.037	-0.960

Observation: Initially the Q-values are near zero because the agent has no experience. As episodes progress, actions that lead toward the goal accumulate higher values, while actions associated with long paths, boundaries, or obstacles receive lower values. This demonstrates the gradual propagation of reward information through the Q-table.

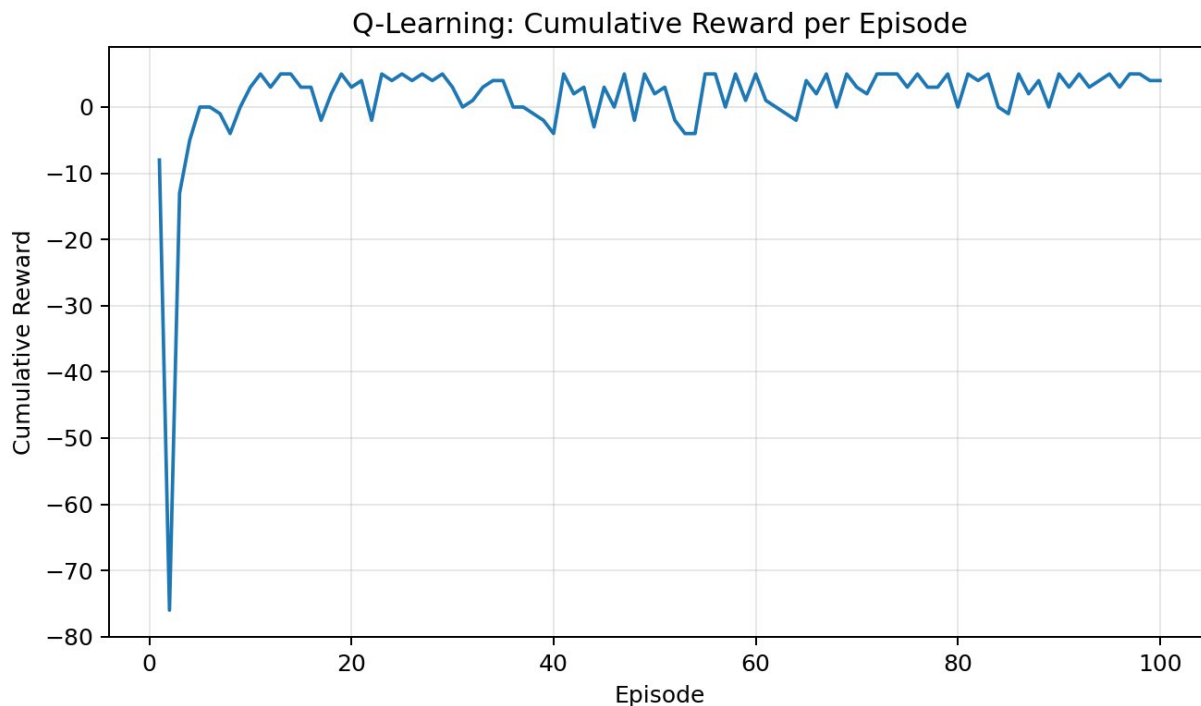
(c) Final Learned Policy and Convergence

Final policy (U = Up, D = Down, L = Left, R = Right, ■ = obstacle, G = goal):

0	1	2	3
D	L	R	D
4	5	6	7
D	■	R	D
8	9	10	11
D	■	D	D

12	13	14	15
R	R	R	G

The learned policy selects the action with the maximum Q-value at each non-terminal state. Because the environment penalizes every step, the agent is encouraged to reach the goal using a short route while avoiding obstacles.



Convergence behaviour: cumulative rewards are initially variable because the agent explores. With repeated interaction, useful Q-values become larger and the agent increasingly follows shorter paths to the goal. Some variation can remain because ϵ -greedy exploration continues throughout training. Therefore, convergence is observed as the rewards become more consistently favourable rather than being perfectly constant.

Conclusion

The Decision Tree experiment demonstrated supervised classification using the Gini Index, including preprocessing, train/test evaluation, tree interpretation, and error analysis. The Q-Learning experiment demonstrated how an agent learns an action-value function through trial and error. Across 100 episodes, the Q-table progressively captured the value of actions that lead toward the goal, resulting in a learned policy for the Grid World.

Python Libraries Used

pandas, NumPy, scikit-learn, matplotlib, and Python's standard random module.